

omniops_db_mcp - Technical Write-up

Tayseer Abdeljaber

OmniOps AI Engineering Internship

What it does

This is an MCP server that lets an AI model query SQL databases. The model never gets a connection to the database itself. It gets four tools, all of them read-only, and the server decides what actually runs.

The first plan was to connect it to one specific data source. After talking it through, the target changed to a server that works with any SQL database instead of being wired to one system, since that is closer to how Bunyan would have to connect to whatever database a customer already has.

How it fits together

```
MCP client (Inspector, Claude Desktop, etc)
    |
    |  stdio
    v
server.py      4 tools
    |
    |  SQLAlchemy
    v
whatever is listed in config.yaml
```

There are three parts.

`config.yaml` is a list of database names and their connection strings. Adding a database means adding a line to this file. Nothing in the code changes.

`server.py` has the four tools and all the query logic. Database connections get created the first time they are needed and then reused, so it is not reconnecting on every call.

The database is whatever the connection string points at. SQLAlchemy handles the differences between Postgres, MySQL and SQLite, so asking for a table list works the same way on all of them.

The four tools

`list_data_sources` takes nothing and returns the database names from the config file.

`list_tables` takes a database name and returns the tables in it.

`describe_table` takes a database name and a table name and returns the columns and their types.

`run_query` takes a database name, a SELECT query, and an optional row limit, and returns the rows as a table.

They are meant to be used in that order. A model that does not know what is in the database can list the tables, look at the columns, and then write a query. That way the server does not have to guess what the model wants.

Arguments get checked before the code runs. The SQL string is capped at 2000 characters and the limit has to be between 1 and 200. Bad input gets rejected before it reaches the database.

Decisions I made and why

Using SQLAlchemy instead of writing a driver per database. If every database needed its own code, supporting a new customer would mean writing new code every time. SQLAlchemy already handles connections and schema inspection for all the major SQL databases, so a new one is a config change. This is the thing the whole “works with any database” idea depends on.

Only allowing SELECT. `run_query` runs the SQL through a parser first and only continues if there is exactly one statement and it is a SELECT. I used a parser instead of just checking whether the string starts with “select”, because `SELECT 1; DROP TABLE people` starts with select. The parser sees two statements there and refuses it.

Applying the row limit myself. Instead of trusting a LIMIT written inside the query, the whole query gets wrapped in another SELECT with the limit added on the outside. That way the caller cannot get

around it. The limit value also gets clamped in code, so asking for a huge number does not widen the cap.

Keeping connection strings out of the code. They only live in `config.yaml`, and `.env` is in `.gitignore`. Nothing is hardcoded.

Logging the queries but not the results. Every tool call gets logged, and `run_query` logs the query text so there is a record of what was run. It does not log the rows that come back. Logging the results would basically make the log file a second copy of the database, which defeats the point if the reason the data is on-premise is that it is not supposed to leave. The logs go to `stderr` because `stdout` is what MCP itself uses to talk to the client.

Testing

`test_server.py` runs 18 checks against the fake database. It checks that each tool returns what it should, that filtering and limits work, and that a bad database name gives a clear error instead of crashing. It also tries five different ways of writing to the database (DROP, DELETE, UPDATE, INSERT, and sneaking a second statement in after a SELECT) and confirms all five get blocked. The last check counts the rows again to make sure nothing actually got deleted along the way.

The test data is made up. It has the same shape as a real table of people and message metadata, but the names are invented and there are no real records in it, so none of this needed real data to build or demo.

What is not done

I have only run it against SQLite. The design should work with Postgres and MySQL because SQLAlchemy handles them the same way, but I have not actually tested that yet. This is the one claim in this document that I am reasoning about rather than something I have shown working, and testing it against a real Postgres database is the next thing to do.

There is no read-only database account yet. Right now writes are blocked by the code only. That is one mistake in the parser away from not working. A real deployment should also connect with a database user that has no write permission, so even if the check failed, nothing could be changed.

LIMIT is not universal. The row cap uses `LIMIT n`, which works on SQLite, Postgres and MySQL, but SQL Server uses different syntax. Supporting it would mean building the limit through SQLAlchemy instead of putting the string together myself.

No query timeout. A slow query could sit there holding a connection. I left this out because SQLite does not really have an equivalent, but a real database would want one.

It only runs locally. The server talks over `stdio`, which means the client starts it as a program on the same machine. There is no login or token because there is no network connection to protect. Running it remotely would need both a network transport and some kind of authentication, which is a separate job.

Open question

I still do not know how Bunyan is supposed to connect to this. MCP does not have webhooks. A client either speaks MCP or it does not. If Bunyan has an MCP client built in, then this needs a network transport and a token. If webhook meant something else, like getting notified when new data arrives, that is a different thing to build. I did not want to guess and build the wrong one, so this is the first thing worth checking before going further.